

Secure Railway Reservation System (SRRS)

Security Assessment & Cross-Language Analysis Report Template

Course: Software Security

Project: Secure Railway Reservation System

Methodology: BUILD → BREAK → ANALYZE → COMPARE → FIX → RETEST → COMPARE → REPORT

Student Name: _____

Student ID: _____

Group: _____

Date: _____

Instructor: _____

Languages Evaluated: Python / Java / C++ / Rust

1. Introduction

Central Question

What is the purpose of this project, why is the same system implemented in multiple programming languages, and how was the BUILD → BREAK → ANALYZE → COMPARE → FIX → RETEST → COMPARE methodology used?

Describe the objective of the security assessment and the overall approach followed.

2. System Overview

2.1 System Description

Central Question

What does the Secure Railway Reservation System do, and what security concerns arise from its main operations?

2.2 Architecture

Include the architecture diagram provided with the project specification.

Central Question

How do the logical components interact, and where are the trust boundaries through which security-relevant data and requests flow?

2.3 Functional Scope

The system covers:

- Authentication
- Train Search
- Seat Availability
- Seat Booking
- Reservation Retrieval

Central Question

What functional behavior must the system provide for a passenger to authenticate, search for a train, view seats, book a seat, and retrieve a reservation?

2.4 Security Requirements

Consider SR-01 to SR-07 from the project specification.

Central Question

What security properties must the system preserve while providing its required functionality?

3. Experimental Setup

Central Question

How were the four implementations built, executed, and tested under comparable conditions?

Language	Version	Build/Run Method	Tools Used
Python			
Java			
C++			
Rust			

4. BUILD — Functional Verification

4.1 Functional Testing

Central Question

Does each implementation correctly perform the required functionality before security testing begins?

Functional Test	Python	Java	C++	Rust
Authentication				
Train Search				
Seat Availability				
Legitimate Booking				
Reservation Retrieval				

Use **PASS / FAIL** and provide brief evidence where necessary.

4.2 BUILD Observations

Central Question

What functional differences, implementation issues, or language-specific observations were identified during the BUILD stage?

5. BREAK — Security Testing

5.1 Security Test Results

Use ST-01 to ST-07 from the Common Test Specification (test_cases.md).

Central Question

Can an attacker cause any implementation to violate one or more of the defined security requirements?

Security Test	Requirement	Python	Java	C++	Rust
ST-01	SR-01				
ST-02	SR-02				
ST-03	SR-03				
ST-04	SR-04				
ST-05	SR-05				
ST-06	SR-06				
ST-07	SR-07				

Use **PASS / FAIL / N/A / NOT DIRECTLY TESTABLE**.

5.2 Attack Evidence

Central Question

What evidence demonstrates that the security attack succeeded or was prevented?

Test ID	Implementation	Attack Input / Request	Observed Output	Result
---------	----------------	------------------------	-----------------	--------

6. ANALYZE — Security & Programming Language Analysis

For each significant security finding, create a separate subsection.

6.X Security Finding

Finding ID: _____

Security Test: _____

Implementation: _____

6.X.1 Security Requirement

Central Question:

What security requirement was affected, and how did the observed behavior satisfy or violate it?

6.X.2 Attack and Impact

Central Question:

What did the attacker do, and what impact did the attack have on the application, data, or security properties?

6.X.3 Code-Level Analysis

Central Question:

What in the code allowed the attack to occur, and what security-relevant logic was missing, weak, or incorrectly implemented?

Include a small relevant code excerpt.

6.X.4 Attack Surface

Central Question:

Where does the attack enter the system, and what makes this part of the system an attack surface?

6.X.5 Programming Language & Security Analysis

Central Question:

How did the programming language influence the vulnerability, its manifestation, or the available mechanisms for preventing or fixing it?

7. COMPARE — Before Fix

7.1 Cross-Language Comparison

Central Question

How did the same security requirement and attack behave across Python, Java, C++, and Rust, and what explains the differences?

Criterion	Python	Java	C++	Rust
Security Requirement				
Attack Behavior				
Impact				
Vulnerable Code				
Attack Surface				
Language Influence				
Fundamental Cause				

7.2 Cross-Language Discussion

Central Question

Was the underlying security problem the same across languages, or did the programming language change its manifestation, exploitability, or consequences?

8. FIX — Security Improvement

For each selected vulnerability:

8.X Security Fix

8.X.1 Security Objective

Central Question:

What security objective must the fix achieve?

8.X.2 Proposed Security Control

Central Question:

What security control or design change should prevent the original attack?

8.X.3 Implementation

Central Question:

How was the proposed security control implemented in the selected programming language?

Include the relevant code excerpt.

8.X.4 Rationale

Central Question:

Why should this change prevent the original attack while satisfying the security requirement?

8.X.5 Functional Impact

Central Question:

Does the security fix preserve the legitimate functionality of the application?

9. RETEST — Security Verification

9.1 Security Retest

Repeat the original security attack after implementing the fix.

Central Question

Does the security fix prevent the original attack and satisfy the affected security requirement?

Finding	Before Fix	Expected After Fix	Actual After Fix	Result
---------	------------	--------------------	------------------	--------

9.2 Functional Retest

Central Question

After applying the security fix, does the legitimate functionality continue to work correctly?

Functional Test	Before Fix	After Fix	Result
Authentication			
Train Search			
Seat Availability			
Legitimate Booking			
Own Reservation Retrieval			

10. COMPARE — After Fix

10.1 Cross-Language Fix Comparison

Central Question

How effectively did each language-specific fix prevent the attack while preserving functionality, and what differences arose from the programming language?

Criterion	Python	Java	C++	Rust
Fix Implemented				
Original Attack Prevented				
Legitimate Functionality Preserved				
Implementation Complexity				
Readability				
Language Support				
Residual Security Concerns				

10.2 Cross-Language Discussion

Central Question

What does the comparison reveal about the relationship between secure design, implementation practices, and programming-language security mechanisms?

11. Overall Security Assessment

Central Question

After testing and remediation, how well does each implementation satisfy the defined security requirements?

Security Requirement	Python	Java	C++	Rust
SR-01				
SR-02				
SR-03				
SR-04				
SR-05				

Security Requirement	Python	Java	C++	Rust
SR-06				
SR-07				

Use **PASS / FAIL / N/A / NOT DIRECTLY TESTABLE**.

12. Lessons Learned

Central Question

What did the experiment demonstrate about security requirements, functional correctness, attack surfaces, secure design, and the influence of programming languages?

Discuss:

- Security requirements versus functional requirements
- Functional correctness versus security correctness
- Identification of attack surfaces
- Security-design problems versus implementation problems
- Programming-language influence on security
- Whether changing programming languages automatically eliminates security problems
- Language features that helped during remediation
- Changes observed between BREAK and RETEST

13. Conclusion

Central Question

What are the most important security findings, cross-language differences, remediation results, and lessons from the complete experiment?

14. References

Central Question

Which technical, academic, standards-based, or other authoritative sources were used to support the security analysis?

Appendix A — Test Evidence

Central Question

What evidence supports the functional tests, security attacks, fixes, and retests reported in this assessment?

Include relevant commands, inputs, outputs, screenshots, test logs, or automated test results.

Appendix B — Relevant Code

Central Question

Which code excerpts provide direct evidence for the security findings and their remediation?

Include only relevant code excerpts and identify the implementation and file/location.

Appendix C — Additional Results

Central Question

What additional technical results support the analysis but were not included in the main report?

Include supplementary test results, tables, logs, or observations where appropriate.